Introduction to Robot Operating System (ROS)

Thumrongsak Kosiyatrakul

Robot Operating System (ROS)

- ROS is not an operating system. It is a **meta**-Operating System
  - ROS is running on top of another operating system (Linux)
- Contains a collection of software, packaging, and building tools
- Suitable for distributed inter-process/inter-machine communication
  - Can be used across networks (TCP/IP)
- Provides tools for data analysis and debugging during run-time
  - Allows debugging under a distributed environment
- Support various languages such as C++ and Python
- **We are going to use `Python` in this class**
  - For CS, it should take about 5 hours to learn enough Python for this class

Robot Operating System (ROS)

- ROS is not an actual OS but has the same functionalities as an OS
  - Supply interprocess communication
  - Signal/interrupt to processes
- All major functionalities are broken up into a number of **nodes**
  - Each node runs as a separate process
  - Nodes communicate with each other using messages
- Matchmaking between nodes is done by the ROS Master (`roscore`)

Robot Operating System (ROS)

- At the lowest level (main):
  - General tools for distributed computing
  - Packaging
  - Building tools
  - API
  - Language binding
- This part is maintained by Willow Garage, Inc and some other developer

Robot Operating System (ROS)

- At higher level (universe):
  - Provides algorithms, hardware abstractions, applications, etc
  - Application Level: playing football
  - Capability Level: Navigation
  - Libraries: OpenCV, Drivers, etc
- Developed and maintained by ROS community
  - The main web site ▸ www.ros.org
  - Questions/Answers Community Support ▸ answers.ros.org

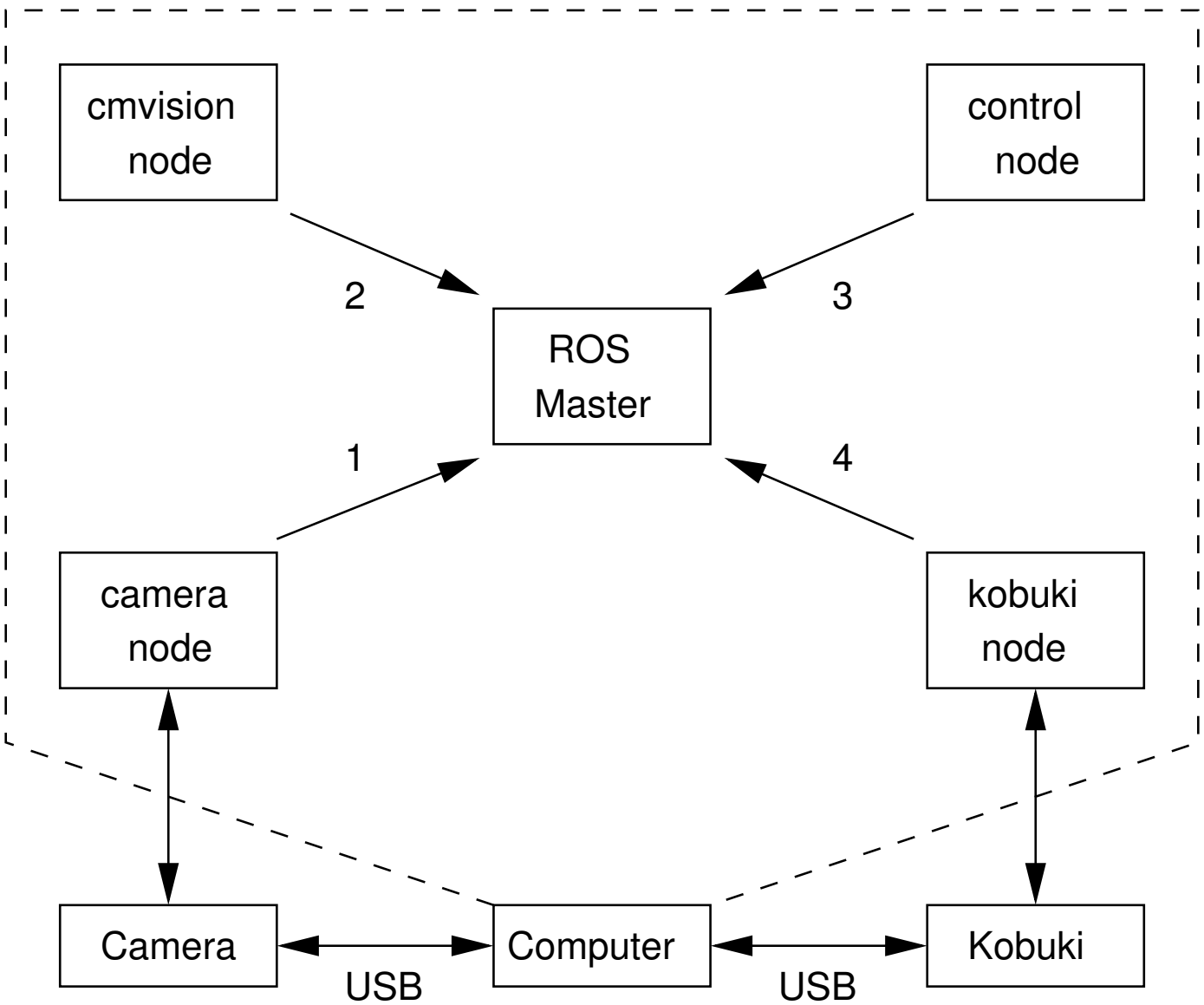
ROS Core

- ROS Master
  - A centralized Remote Procedure Call (RPC) server
  - Negotiates communication connections between nodes
  - Registers names
- `rosout`: network standard output
  - Can show output of nodes running on different machines over networks

ROS Communication Protocol

- ROS Topics
  - Asynchronous communication (stream)
  - Strongly Typed
    - Help hiding underly byte stream
    - Think about `struct` in C
  - Can have one or more **publishers** in the same topic
  - Can have one or more **subscribers** in the same topic

How ROS Works?



How ROS Works?

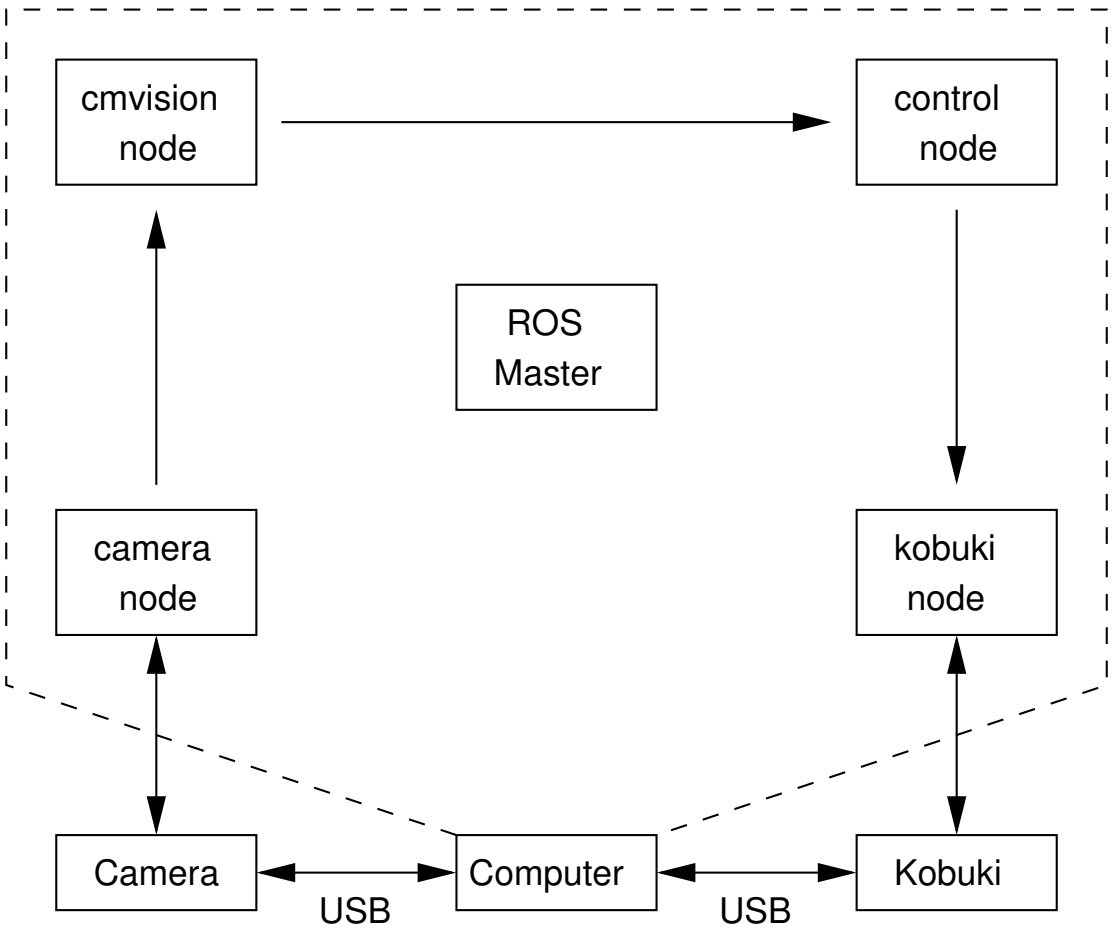
- **camera node**
  - Performs hardware abstraction to support all kinds of USB cameras
    - Turn the data from camera into a standard image format
  - Tells the ROS Master
    - I wants to publish images on topic **image**
- **cmvision node**
  - Performs objects detection based on blob of colors
    - Turn an image into an array of coordinate of detected objects
  - Tells the ROS Master
    - I want to subscribe to receive images from the topic **image**
    - I also want to publish blobs on topic **blobs**

How ROS Works?

- **control node**
  - Make decisions to turn left/right based on an array of coordinates
  - Tells the ROS Master
    - I want to subscribe to receive blobs from the topic **blobs**
    - I also want to publish velocities on topic **velocity**
- **kobuki node**
  - Performs hardware abstractions
    - Motor controller, motor feedback, sensors
  - Tells the ROS Master
    - I want to subscribe to receive velocities from the topic **velocity**

Note: the order does not matter

How ROS Works?



ROS Master sets up communication among nodes based on their requests.

ROS Nodes

- A **node** is a process that performs some computations.
- We typically divide the entire software functionality into different modules
  - Single Node
    - Color image to array of coordinate in one program
  - Multiple Nodes:
    - Color image to black and white
    - Black and white to coordinate
- Nodes are combined together into a graph and communicate with one another using streaming topics
- These nodes are meant to operate at a fine-grained scale
  - A robot control system usually comprise of many nodes.

ROS Node Examples

- **gscam node**:
  - Communicates with integrated web camera at the hardware level
  - Captures images and publishes on topic **image**
  - **Note** that a video is simply a series of images
- **cmvision node**:
  - Subscribes to the topic **image**
  - Tries to find blobs of specific color for every image received
  - Publishes locations and sizes of blobs on topic **blobs**
- **control node**:
  - Subscribes to the topic **blob**
  - Decides what to do
  - Publishes velocities to move robot
- **kobuki node**:
  - Subscribes to the topic **velocity**
  - Tells kobuki robot what to do based on velocities received

ROS Topics

- Topics are named buses over which nodes exchange messages
  - There can be hundreds of topics at a time
- Topics have anonymous publish/subscribe semantics
  - A subscriber does not care which node published the data it receives
  - A publisher does not care which node subscribes to the data it publishes.
- There can be multiple publishers and subscribers to a topic
- Each topic is **strongly** typed by the ROS message it transports
- Transport is done using TCP or UDP
  - Can be between multiple machines over networks

ROS Messages

- Nodes communicate with each other by publishing messages to topics
  - Publishers publish data to topics
  - Subscribers consume data from topics
- A message is a simple data structure, comprising typed fields.
- Similarly to data type in programming:
  - `std_msgs/Int32`
  - `std_msgs/Float32`
  - `std_msgs/String`
  - `std_msgs/Empty`

  The numbers at the end indicates number of bits
- Reference to standard messages (`std_msgs`) can be found at `http://wiki.ros.org/std_msgs`